

TASK 1: TEST-DRIVEN DEVELOPMENT FOR EVENODD NUMBER VALIDATOR

AI PROMPT USED: GENERATE UNITTEST TEST CASES FOR A FUNCTION CALLED IS_EVEN(N) THAT CHECKS IF AN INTEGER N IS EVEN.

INCLUDE TESTS FOR POSITIVE EVEN, ODD, ZERO, NEGATIVE EVEN, NEGATIVE ODD, AND LARGE NUMBERS. INPUT MUST BE INTEGER ONLY.

AI GENERATED TEST CASES:

```
import unittest class
testiseven(unittest.TestCase): def
test_even_positive(self):
self.assertTrue(is_even(2)) def
test_odd_positive(self):
self.assertFalse(is_even(7)) def
test_zero(self):
self.assertTrue(is_even(0)) def
test_negative_even(self):
self.assertTrue(is_even(-4)) def
test_negative_odd(self):
self.assertFalse(is_even(-9)) def
test_large_even(self):
self.assertTrue(is_even(1000000))
def test_large_odd(self):
self.assertFalse(is_even(999999)) if
__name__ == '__main__': unittest.main()
```

IMPLEMENTED FUNCTION:

```

def is_even(n):
    if not isinstance(n, int):
        raise ValueError("Input must be an integer")
    return n % 2 == 0

```

TERMINAL OUTPUT - RUNNING TESTS:

```
$ PYTHON -m unittest test_is_even.py ....
```

```
----- RAN 7 TESTS IN 0.001S
```

OK

TASK 2: TEST-DRIVEN DEVELOPMENT FOR STRING CASE CONVERTER

AI PROMPT USED: GENERATE PYTEST TEST CASES FOR TWO FUNCTIONS: TO_UPPERCASE(TEXT) AND TO_LOWERCASE(TEXT). HANDLE EMPTY STRINGS, MIXED CASE, NUMBERS, NONE. RAISE VALUEERROR FOR INVALID INPUTS LIKE NONE OR NONSTRINGS.

AI GENERATED TEST CASES:

```

import pytest

def test_to_uppercase_normal():
    assert to_uppercase("AI CODING") == "AI CODING"

def test_to_uppercase_empty():
    assert to_uppercase("") == ""

def test_to_uppercase_mixed():
    assert to_uppercase("TEST123") == "TEST123"

def test_to_uppercase_none():
    with pytest.raises(ValueError):
        to_uppercase(None)

def test_to_lowercase_normal():
    assert to_lowercase("TEST") == "test"

def test_to_lowercase_empty():
    assert to_lowercase("") == ""

def test_to_lowercase_numbers():
    assert to_lowercase("ABC123") == "abc123"

```

IMPLEMENTED FUNCTIONS:

```
def to_uppercase(text):    if text is None or not isinstance(text, str):
    raise ValueError("INPUT MUST BE A STRING")    return text.upper()
def to_lowercase(text):    if text is None or not isinstance(text, str):        raise
    ValueError("INPUT MUST BE A STRING")
    return text.lower()
```

TERMINAL OUTPUT - RUNNING TESTS:

```
$ PYTEST TEST_STRING_CASE.PY -V TEST_STRING_CASE.PY::TEST_TO_UPPERCASE_NORMAL PASSED
TEST_STRING_CASE.PY::TEST_TO_UPPERCASE_EMPTY PASSED
TEST_STRING_CASE.PY::TEST_TO_UPPERCASE_MIXED PASSED
TEST_STRING_CASE.PY::TEST_TO_UPPERCASE_NONE PASSED
TEST_STRING_CASE.PY::TEST_TO_LOWERCASE_NORMAL PASSED
TEST_STRING_CASE.PY::TEST_TO_LOWERCASE_EMPTY PASSED
TEST_STRING_CASE.PY::TEST_TO_LOWERCASE_NUMBERS PASSED ===== 7
PASSED IN 0.02S =====
```

TASK 3: TEST-DRIVEN DEVELOPMENT FOR LIST SUM CALCULATOR

AI PROMPT USED: CREATE UNITTEST TESTS FOR SUM_LIST_NUMBERS(LST) THAT SUMS NUMERIC ELEMENTS IN A LIST, IGNORES NON-NUMBERS, HANDLES EMPTY LISTS (RETURN 0), NEGATIVES.

AI GENERATED TEST CASES:

```

import unittest class
TESTSUMLIST(unittest.TestCase): def
TEST_NORMAL_LIST(self):
    self.assertEqual(sum_list_numbers([1, 2, 3]), 6) def
TEST_EMPTY_LIST(self):
    self.assertEqual(sum_list_numbers([]), 0) def
TEST_NEGATIVES(self):
    self.assertEqual(sum_list_numbers([-1, 5, -4]), 0) def
TEST_MIXED(self):
    self.assertEqual(sum_list_numbers([2, 'A', 3]), 5)
def test_all_non_numeric(self):
    self.assertEqual(sum_list_numbers(['A', 'B']), 0) def
TEST_FLOATS(self):
    self.assertEqual(sum_list_numbers([1.5, 2.5]), 4.0)

```

IMPLEMENTED FUNCTION:

```

def sum_list_numbers(lst):
    total = 0
    for item in lst:
        if isinstance(item, (int, float)):
            total += item
    return total

```

TERMINAL OUTPUT - RUNNING TESTS:

```

$ python -m unittest test_sum_list.py .....
6 tests in 0.000s

```

OK

TASK 4: TEST CASES FOR STUDENT RESULT CLASS

AI PROMPT USED: GENERATE TESTS FOR STUDENTRESULT CLASS WITH add_marks(mark), calculate_average(), get_result().

MARKS 0-100, AVG >=40 PASS ELSE FAIL. VALIDATE MARKS RANGE.

AI GENERATED TEST CASES:

```
import unittest
class TestStudentResult(unittest.TestCase):
    def test_pass_case(self):
        sr = StudentResult()
        sr.add_marks(60)
        sr.add_marks(70)
        sr.add_marks(80)
        self.assertEqual(sr.calculate_average(), 70)
        self.assertEqual(sr.get_result(), "PASS")
    def test_fail_case(self):
        sr = StudentResult()
        sr.add_marks(30)
        sr.add_marks(35)
        sr.add_marks(40)
        self.assertEqual(sr.calculate_average(), 35)
        self.assertEqual(sr.get_result(), "FAIL")
    def test_invalid_mark(self):
        sr = StudentResult()
        with self.assertRaises(ValueError):
            sr.add_marks(-10)
    def test_no_marks(self):
        sr = StudentResult()
        self.assertEqual(sr.calculate_average(), 0)
        self.assertEqual(sr.get_result(), "FAIL")
```

IMPLEMENTED CLASS:

```
class StudentResult:
    def __init__(self):
```

```

    SELF.MARKS = []      DEF ADD_MARKS(SELF,
MARK):      IF NOT ISINSTANCE(MARK, (INT, FLOAT)) OR
MARK < 0 OR MARK > 100:
    RAISE ValueError("MARK MUST BE BETWEEN 0 AND 100")
SELF.MARKS.APPEND(MARK)  DEF CALCULATE_AVERAGE(SELF):  IF
NOT SELF.MARKS:      RETURN 0      RETURN SUM(SELF.MARKS) /
LEN(SELF.MARKS)  DEF GET_RESULT(SELF):
    AVG = SELF.CALCULATE_AVERAGE()  RETURN
"PASS" IF AVG >= 40 ELSE "FAIL"

```

TERMINAL OUTPUT - RUNNING TESTS:

```

$ PYTHON -M unittest test_student.py ..... RAN 4
TESTS IN 0.001S

OK

```

TASK 5: TEST-DRIVEN DEVELOPMENT FOR USERNAME VALIDATOR

AI PROMPT USED: UNITTEST TESTS FOR IS_VALID_USERNAME(USERNAME): MIN 5 CHARS, ALPHANUMERIC ONLY, NO SPACES.

AI GENERATED TEST CASES:

```

import unittest class
testusername(unittest.TestCase):  DEF
test_valid(self):
self.assertTrue(is_valid_username("USER01"
))  DEF test_too_short(self):
self.assertFalse(is_valid_username("A"))
DEF test_space(self):
    self.assertFalse(is_valid_username("USER NAME"))  DEF
test_special_chars(self):

```

```

        SELF.ASSERTFALSE(IS_VALID_USERNAME("USER@123"))    DEF
TEST_VALID_LONG(SELF):
SELF.ASSERTTRUE(IS_VALID_USERNAME("USERNAME12345"))    DEF
TEST_EMPTY(SELF):
    SELF.ASSERTFALSE(IS_VALID_USERNAME(""))

```

IMPLEMENTED FUNCTION:

```

IMPORT RE DEF IS_VALID_USERNAME(USERNAME): IF NOT ISINSTANCE(USERNAME,
STR):
    RETURN FALSE IF LEN(USERNAME)
< 5:
    RETURN FALSE IF ' ' IN USERNAME: RETURN FALSE RETURN
BOOL(RE.MATCH(R'^[A-ZA-Z0-9]+$', USERNAME))

```

TERMINAL OUTPUT - RUNNING TESTS:

```
$ PYTHON -M unittest test_username.py .....
```

```
----- RAN 6 TESTS IN 0.002S
```

```
OK
```